

COMPUTER ARCHITECTURE



Hamid Sarbazi-Azad

Department of Computer Science and Engineering
Sharif University of Technology,
Tehran, Iran

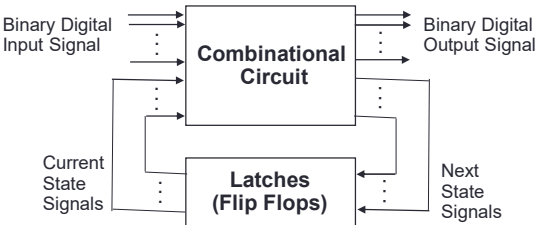
Overview of Logic Circuits

Logic Circuits can be:

→ Combinational: output logic value depends only on the input logic values.



→ Sequential: output logic value depends on the input values and the state (stored information) of the circuit.



(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #2: Overview of Logic Circuits3

Combinational Circuits

Combinational circuits can be described by:

- Truth Table
- Boolean Function
- Karnaugh Map

Logic gates are the basic combinational elements.

They are used to design complex combinational logic blocks such as:

- Decoder
- Encoder
- Multiplexer
- DeMultiplexer
- Adders
- ...

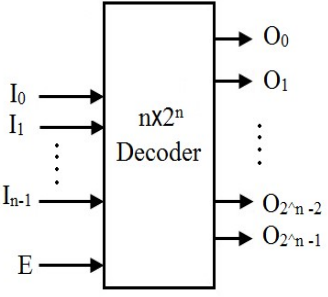
Name	Symbol	Truth Table															
AND	 $A \cdot B$ or AB	<table><thead><tr><th>A</th><th>B</th><th>AB</th></tr></thead><tbody><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></tbody></table>	A	B	AB	0	0	0	0	1	0	1	0	0	1	1	1
A	B	AB															
0	0	0															
0	1	0															
1	0	0															
1	1	1															
OR	 $A + B$	<table><thead><tr><th>A</th><th>B</th><th>A+B</th></tr></thead><tbody><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></tbody></table>	A	B	A+B	0	0	0	0	1	1	1	0	1	1	1	1
A	B	A+B															
0	0	0															
0	1	1															
1	0	1															
1	1	1															
Not	 A' or $\bar{A}$	<table><thead><tr><th>A</th><th>$\bar{A}$</th></tr></thead><tbody><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></tbody></table>	A	$\bar{A}$	0	1	1	0									
A	$\bar{A}$																
0	1																
1	0																
Buffer	 A	<table><thead><tr><th>A</th><th>A</th></tr></thead><tbody><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></tbody></table>	A	A	0	0	1	1									
A	A																
0	0																
1	1																
NAND Not And	 $(AB)'$ or $\overline{AB}$	<table><thead><tr><th>A</th><th>B</th><th>$\overline{AB}$</th></tr></thead><tbody><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></tbody></table>	A	B	$\overline{AB}$	0	0	1	0	1	1	1	0	1	1	1	0
A	B	$\overline{AB}$															
0	0	1															
0	1	1															
1	0	1															
1	1	0															
NOR Not Or	 $(A+B)'$ or $\overline{A+B}$	<table><thead><tr><th>A</th><th>B</th><th>$\overline{A+B}$</th></tr></thead><tbody><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></tbody></table>	A	B	$\overline{A+B}$	0	0	1	0	1	0	1	0	0	1	1	0
A	B	$\overline{A+B}$															
0	0	1															
0	1	0															
1	0	0															
1	1	0															
XOR Exclusive OR	 $A \oplus B$ or $A'B + AB'$	<table><thead><tr><th>A</th><th>B</th><th>$A \oplus B$</th></tr></thead><tbody><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></tbody></table>	A	B	$A \oplus B$	0	0	0	0	1	1	1	0	1	1	1	0
A	B	$A \oplus B$															
0	0	0															
0	1	1															
1	0	1															
1	1	0															
XNOR Exclusive NOR or Equivalence	 $(A \oplus B)'$ or $A \odot B$	<table><thead><tr><th>A</th><th>B</th><th>$A \odot B$</th></tr></thead><tbody><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></tbody></table>	A	B	$A \odot B$	0	0	1	0	1	0	1	0	0	1	1	1
A	B	$A \odot B$															
0	0	1															
0	1	0															
1	0	0															
1	1	1															

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #2: Overview of Logic Circuits4

Decoder

An $n \times 2^n$ Decoder (or n-DEC) has an Enable input line and n input and 2^n output lines.

If Enable is active then the output line addressed by the input lines is activated.



E	I_{n-1}	...	I_1	I_0	O_0	O_1	...	O_{2^n-2}	O_{2^n-1}
0	x	...	x	x	0	0	...	0	0
1	0	...	0	0	1	0	...	0	0
1	0	...	0	1	0	1	...	0	0
		...					...		
1	1	...	1	0	0	0	...	1	0
1	1	...	1	1	0	0	...	0	1

2

(c) Hamid Sarbazi-Azad

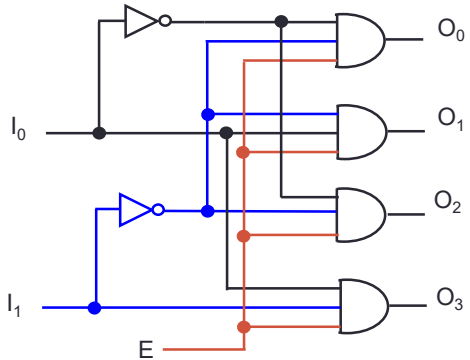
Computer Architecture -- Lecture #2: Overview of Logic Circuits

5

Decoder

Example:

The internal logic and truth table of a 2x4 Decoder.



E	I ₁	I ₀	O ₀	O ₁	O ₂	O ₃
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	x	x	0	0	0	0

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

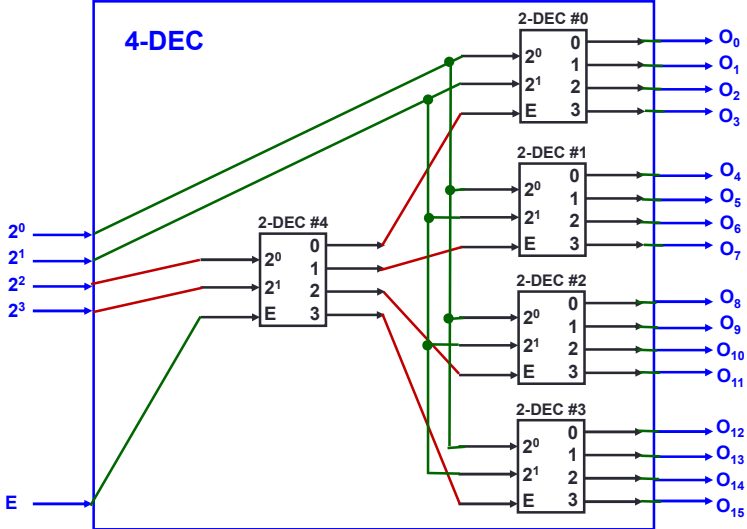
6

Decoder: Extension

Example:

Design a 4-DEC with five 2-DECs.

(High-order Interleaving)



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

7

Decoder: Extension

Example: Design a 4-DEC with five 2-DECs. (Low-order Interleaving)

4-DEC

In how many ways

Answer: 4!

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

8

Decoder: Extension

Example: Design a 3-DEC with four 1-DECs and one 2-DEC.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

9

Decoder: Extension

Example: Design a 3-DEC with four 1-DECs and one 2-DEC.
Alternative design

The diagram illustrates a 3-DEC extension. It features a 2-DEC and four 1-DECs. The 2-DEC has inputs 2^0 , 2^1 , 2^2 , and E , and outputs O_0 , O_1 , O_2 , O_3 . The 1-DECs are labeled 1-DEC #0, 1-DEC #1, 1-DEC #2, and 1-DEC #3. Each 1-DEC has inputs 2^0 , 2^1 , 2^2 , and E , and outputs O_4 , O_5 , O_6 , O_7 . The 2-DEC's E is connected to the 2^2 input of 1-DEC #3. The 2-DEC's 2^0 and 2^1 are connected to the 2^1 and 2^0 inputs of 1-DEC #3. The 2-DEC's 2^2 is connected to the 2^2 input of 1-DEC #2. The 2-DEC's 2^3 is connected to the 2^2 input of 1-DEC #1. The 2-DEC's 2^0 is connected to the 2^0 input of 1-DEC #0. The 2-DEC's 2^1 is connected to the 2^0 input of 1-DEC #1. The 2-DEC's 2^2 is connected to the 2^0 input of 1-DEC #2. The 2-DEC's 2^3 is connected to the 2^0 input of 1-DEC #3.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

10

Decoder: Implementing Logic Functions

All Minterms are generated by a decoder. So, we can easily implement any logic function with decoder and OR gate using its truth table.

Example: Implement $f(x,y,x) = (xy' + x'y)z$ using a decoder.

z	y	x	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

The diagram shows two implementations of the logic function $f(x,y,x) = (xy' + x'y)z$. The top implementation uses a 3-DEC with inputs x , y , z and enable 1 . The 3-DEC has outputs 0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 . The function f is implemented by ORing outputs 5 and 6 . The bottom implementation uses a 2-DEC with inputs x , y and enable z . The 2-DEC has outputs 0 , 1 , 2 , 3 . The function f is implemented by ORing outputs 1 and 2 .

Better design

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

11

Decoder: Implementing Logic Functions

Example: Implement function $f(w,x,y,z) = m(1,3,4,5,8,10,11,12,13,14,15)$ using a 4-DEC.

The diagram illustrates two ways to implement a logic function using a 4-to-16 decoder. On the left, a 4-DEC block has inputs z (2^0), y (2^1), x (2^2), and w (2^3), and an enable input E set to 1. Its 16 outputs are numbered 0 to 15. A large OR gate takes 11 of these outputs (1, 3, 4, 5, 8, 10, 11, 12, 13, 14, 15) as inputs to produce the function f . On the right, the same 4-DEC block is shown, but the OR gate is smaller and more compact, also taking the same 11 decoder outputs as inputs to produce f . A blue arrow labeled 'Better design' points from the first circuit to the second.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

12

Multiplexer

An $n \times m \times d$ Multiplexer ($m=2^n$) has n input select lines and m input data lines (each of d bits) and put the input data addressed by the input select lines on the output data (d bits).

The diagram shows an $n \times m \times d$ Multiplexer (MUX). It has m input data lines, each d bits wide, indexed from 0 to $2^n - 1$. It also has n select lines, indexed from 2^{n-1} down to 2^0 . The output is a single d -bit line.

Select lines	Output
$2^{n-1} \dots 2^1 2^0$	
0 0 ... 0 0	I_0
0 0 ... 0 1	I_1
0 0 ... 1 0	I_2
.	.
.	.
1 1 ... 1 0	$I_{2^n - 2}$
1 1 ... 1 1	$I_{2^n - 1}$

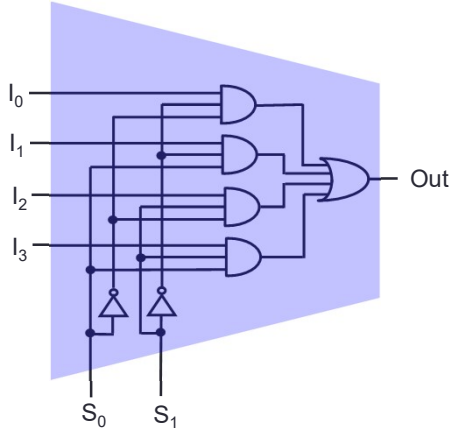
(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

13

Multiplexer

Example: Internal logic of a 2x4x1 MUX.



The diagram illustrates the internal logic of a 2x4x1 multiplexer. It features four data inputs labeled I_0, I_1, I_2, I_3 and two select inputs labeled S_0 and S_1 . The select inputs are connected to two inverters. The outputs of these inverters are connected to four 2-input AND gates. Each data input I_i is connected to one input of each AND gate. The other input of each AND gate is connected to either S_0 or $\overline{S_0}$ (the output of the inverter). Specifically, the AND gates are configured as follows: AND 0 takes I_0 and S_0 ; AND 1 takes I_1 and S_0 ; AND 2 takes I_2 and $\overline{S_0}$; AND 3 takes I_3 and $\overline{S_0}$. The outputs of all four AND gates are connected to a single 4-input OR gate, which produces the final output labeled 'Out'.

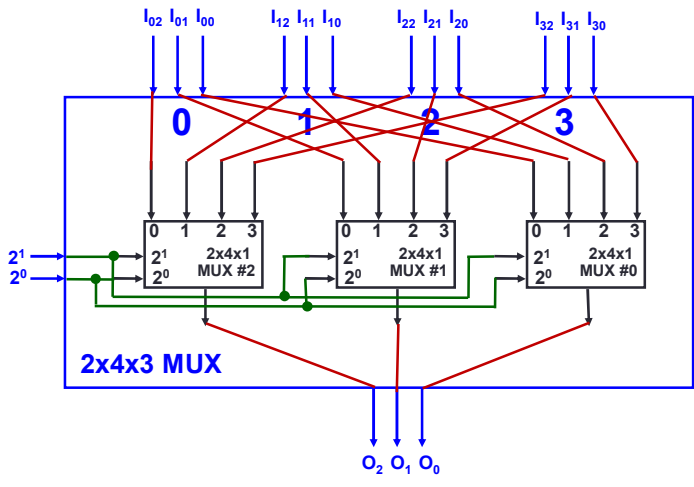
(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

14

Multiplexer: Extension of Data Width

Example: Design a 2x4x3 MUX using 2x4x1 MUXs.



The diagram shows a 2x4x3 multiplexer constructed from three 2x4x1 multiplexers, labeled MUX #2, MUX #1, and MUX #0. Each 2x4x1 MUX has four data inputs (0, 1, 2, 3), two select inputs (2^1 and 2^0), and one output. The three 2x4x1 MUXs are connected in parallel, sharing the same select inputs 2^1 and 2^0 . The data inputs for each MUX are connected to the 12 data inputs of the 2x4x3 MUX: MUX #2 takes inputs I_{02}, I_{01}, I_{00} ; MUX #1 takes inputs I_{12}, I_{11}, I_{10} ; and MUX #0 takes inputs I_{22}, I_{21}, I_{20} . The outputs of the three MUXs are connected to the three outputs of the 2x4x3 MUX, labeled O_2, O_1, O_0 . Red lines indicate the internal connections between the data inputs of the 2x4x3 MUX and the data inputs of the three 2x4x1 MUXs. The entire assembly is enclosed in a blue box labeled '2x4x3 MUX'.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

15

Multiplexer: Extension of Inputs

Example: Design a 4x16x1 MUX using 2x4x1 MUXs. (High-order Interleaving)

The diagram illustrates the high-order interleaving method for constructing a 4x16x1 multiplexer. It features four 2x4x1 multiplexers (MUX #0, #1, #2, #3) arranged horizontally. Each 2x4x1 MUX has four data inputs (0, 1, 2, 3), two select inputs (2^1 and 2^0), and one output. The 16 data inputs of the 4x16x1 MUX are labeled I_0 through I_{15} at the top. These inputs are grouped into four sets of four, each connected to the data inputs of one of the 2x4x1 MUXs. Specifically, I_0 to I_3 connect to MUX #0, I_4 to I_7 to MUX #1, I_8 to I_{11} to MUX #2, and I_{12} to I_{15} to MUX #3. The two select inputs, 2^3 and 2^2 , are connected to the 2^1 and 2^0 select inputs of all four 2x4x1 MUXs. The outputs of the four 2x4x1 MUXs are then connected to the data inputs (0, 1, 2, 3) of a final 2x4x1 MUX (MUX #3) at the bottom. The select inputs of this final MUX are 2^1 and 2^0 , which are connected to the 2^3 and 2^2 select inputs of the top-level 4x16x1 MUX. The output of the final MUX is the output of the 4x16x1 MUX.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

16

Multiplexer: Extension of Inputs

Example: Design a 4x16x1 MUX using 2x4x1 MUXs. (Low-order Interleaving)

The diagram illustrates the low-order interleaving method for constructing a 4x16x1 multiplexer. It features four 2x4x1 multiplexers (MUX #0, #1, #2, #3) arranged horizontally. Each 2x4x1 MUX has four data inputs (0, 1, 2, 3), two select inputs (2^1 and 2^0), and one output. The 16 data inputs of the 4x16x1 MUX are labeled I_0 through I_{15} at the top. These inputs are grouped into four sets of four, each connected to the data inputs of one of the 2x4x1 MUXs. Specifically, I_0 to I_3 connect to MUX #0, I_4 to I_7 to MUX #1, I_8 to I_{11} to MUX #2, and I_{12} to I_{15} to MUX #3. The two select inputs, 2^3 and 2^2 , are connected to the 2^1 and 2^0 select inputs of all four 2x4x1 MUXs. The outputs of the four 2x4x1 MUXs are then connected to the data inputs (0, 1, 2, 3) of a final 2x4x1 MUX (MUX #3) at the bottom. The select inputs of this final MUX are 2^1 and 2^0 , which are connected to the 2^3 and 2^2 select inputs of the top-level 4x16x1 MUX. The output of the final MUX is the output of the 4x16x1 MUX.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

17

Multiplexer: Extension of Inputs

Example: Design a 4x16x1 MUX using two 2x4x1 MUXs, one 3x8x1 MUX, and two 1x2x1 MUXs.

4x16x1 MUX

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

18

Multiplexer: Implementing Logic Functions

In a MUX, each Minterm is ANDed with corresponding input and then the output of all ANDs are ORed to generate the final output. So, we can directly use any **n-input function's** truth table to implement the function using an **$n \times 2^n \times 1$ MUX**.

Example: Implement $f(x,y,z) = m(1,3,4,7)$ using a multiplexer.

xyz	f
000	0
001	1
010	0
011	1
100	1
101	0
110	0
111	1

3x8x1 MUX

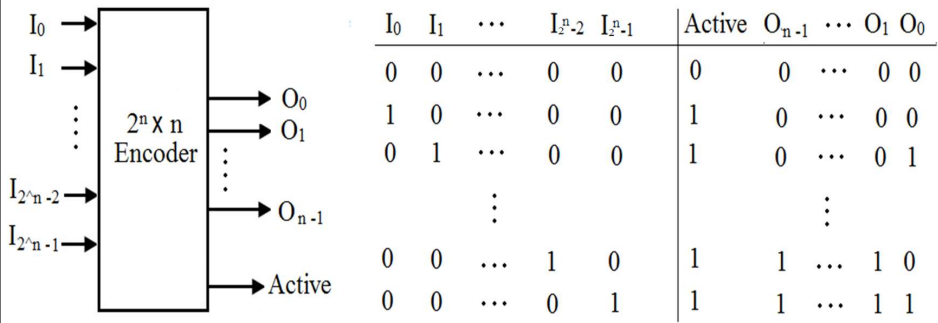
2x4x1 MUX

Better design, if we have the invert of one input. Assume we have x' .

Encoder

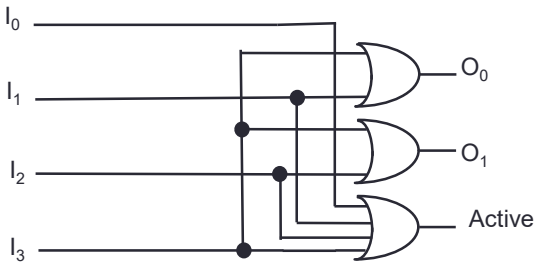
A $2^n \times n$ Encoder has 2^n input lines and n output lines and Active output line.

If none of inputs are active, the Active output is 0. Otherwise, Active output is 1 and the output lines show the address of the active input line (assuming that only one input line is active).



Encoder

Example: Logic diagram of a 4x2 Encoder.



What happens if both I_1 and I_2 are active?
 $O_1 O_0$ will be 11 that is input I_3 is active !!!!!

Priority Encoders can resolve this problem when more than one inputs are active.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

21

Priority Encoder

A $2^n \times n$ Priority Encoder has 2^n input lines and n output lines and **Active** output line.

If none of inputs are active, the Active output is 0. Otherwise, Active output is 1 and the output lines show the address of the active input line with highest priority.

I_0

I_1

$\vdots$

I_{2^n-2}

I_{2^n-1}

$2^n \times n$
Priority
Encoder

O_0

O_1

$\vdots$

O_{n-1}

Active

I_0	I_1	$\dots$	I_{2^n-2}	I_{2^n-1}	Active	O_{n-1}	$\dots$	O_1	O_0
0	0	$\dots$	0	0	0	0	$\dots$	0	0
1	x	$\dots$	x	x	1	0	$\dots$	0	0
0	1	$\dots$	x	x	1	0	$\dots$	0	1
		$\vdots$					$\vdots$		
0	0	$\dots$	1	x	1	1	$\dots$	1	0
0	0	$\dots$	0	1	1	1	$\dots$	1	1

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

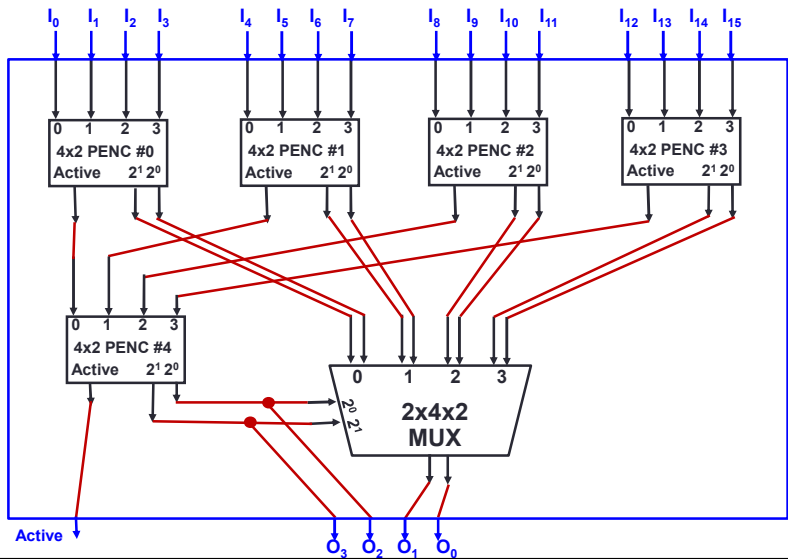
22

Priority Encoder

Example: Logic diagram of a 4x2 Priority Encoder.

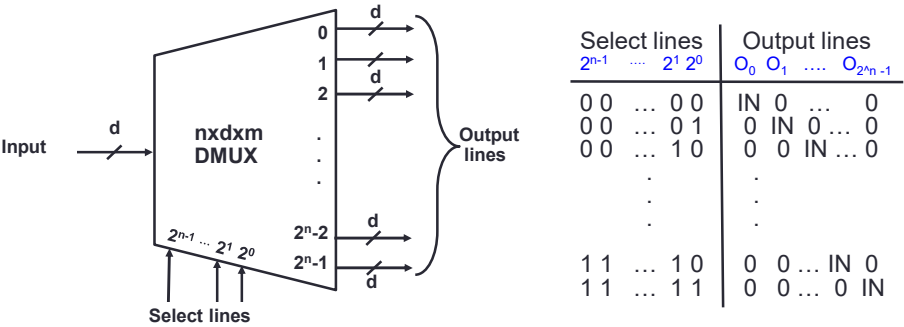
Priority Encoder: Extension

Example: Design a 16x4 Priority Encoder using 4x2 Priority Encoders.



DeMultiplexer

An $n \times d \times m$ DeMultiplexer ($m=2^n$) has n input select lines and a d -bit input and m d -bit output data lines. It puts the input data on the line addressed by select lines.



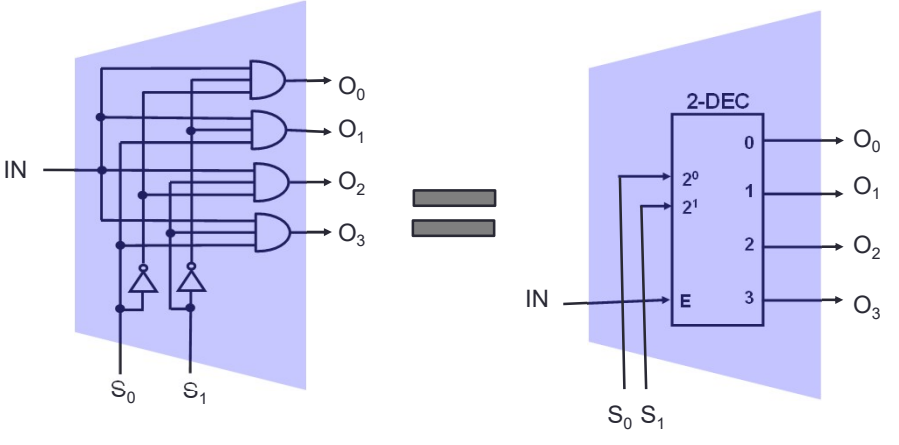
(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

25

DeMultiplexer

Example: Internal logic of a 2x1x4 DMUX.



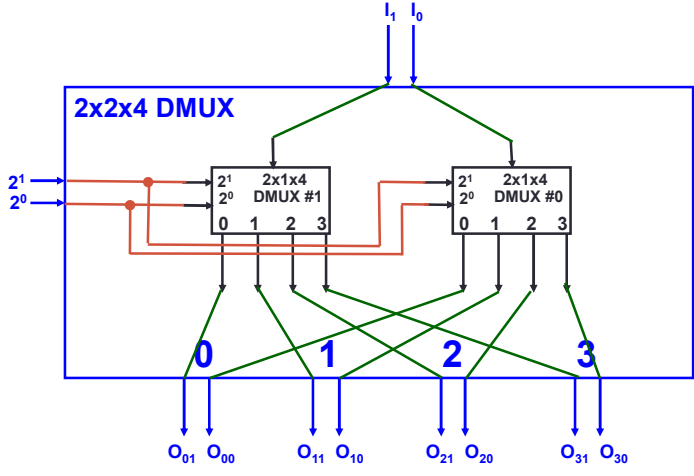
(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

26

DeMultiplexer: Extension of Data Width

Example: Design a 2x2x4 DMUX using 2x1x4 DMUXs.



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

27

DeMultiplexer: Extension of Outputs

Example: Design a 4x1x16 DMUX using 2x1x4 DMUXs. (High-order Interleaving)

The diagram illustrates a 4x1x16 DeMultiplexer (DMUX) constructed using four 2x1x4 DMUXs (labeled #0, #1, #2, #3). The input is a 4-bit signal (2³, 2², 2¹, 2⁰). The output is a 16-bit signal (O₀ to O₁₅). The diagram shows the internal wiring and the resulting output lines, demonstrating the high-order interleaving technique.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

28

DeMultiplexer: Extension of Outputs

Example: Design a 4x1x16 DMUX using 2x1x4 DMUXs. (Low-order Interleaving)

The diagram illustrates a 4x1x16 DeMultiplexer (DMUX) constructed using four 2x1x4 DMUXs (labeled #0, #1, #2, #3). The input is a 4-bit signal (2³, 2², 2¹, 2⁰). The output is a 16-bit signal (O₀ to O₁₅). The diagram shows the internal wiring and the resulting output lines, demonstrating the low-order interleaving technique.

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #2: Overview of Logic Circuits29

Sequential Circuits

A Sequential Logic Circuit comprises of two parts:

- 1) Combinational to generate output and next states
- 2) Memory elements to keep the state of the circuit

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #2: Overview of Logic Circuits30

Flip Flops

The basic memory elements to store one bit is a Flip Flop (FF).

The simplest flip flop can be made using two back to back NOTs:

0-state

1-state

To be able to change its state, we need some more inputs and output lines to set it to desirable states, and read its state externally.

Example: RS FF (also known as RS Latch).

S	R	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	forbidden

$$Q_{t+1} = S + R'Q_t$$

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

31

Clocked Flip Flops

To have a predictable system in a large digital circuit, operations of individual flip flops are synchronized to a clock pulse.

The flip flop has change state only when there is a clock pulse appearing at the clock line.

We call above flip flop a Clocked RS FF, and show it symbolically as

operates when clock is high

operates when clock is low

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

32

RS Flip Flop with Preset/Clear Inputs

active-high
Clock, Preset
and Clear inputs

active-low
Clock, Preset
and Clear inputs

active-high Clock
active-low Preset
and Clear inputs

active-low Clock
active-high Preset
and Clear inputs

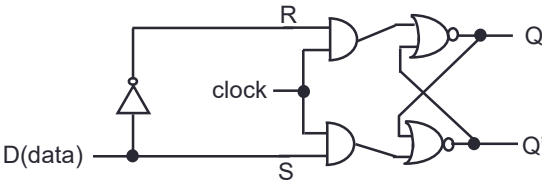
(c) Hamid Sarbazi-Azad

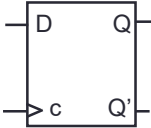
Computer Architecture -- Lecture #2: Overview of Logic Circuits

33

Data Flip Flop (D-FF)

D-FF is mainly used for data storage and is made from RS-FF where forbidden input values are not allowed using an inverter between the inputs.





D	Q_{t+1}
0	0
1	1

$$Q_{t+1} = D$$

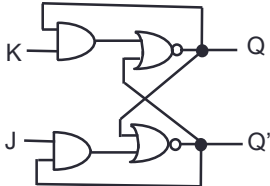
(c) Hamid Sarbazi-Azad

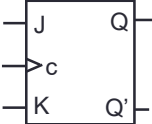
Computer Architecture -- Lecture #2: Overview of Logic Circuits

34

JK Flip Flop

JK-FF is like RS_FF except that the forbidden inputs in RS_FF is possible for JK-FF.

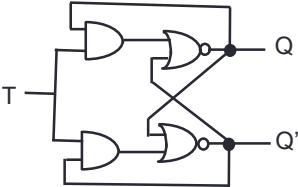


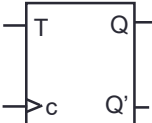


J	K	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	Q_t'

$$Q_{t+1} = JQ_t' + K'Q_t$$

Toggle flip flop (T-FF) is a JK-FF where both J and K are joined and named T.





T	Q_{t+1}
0	Q_t
1	Q_t'

$$Q_{t+1} = T \text{ xor } Q_t$$

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

35

Edge-Triggered Flip Flops

Characteristics

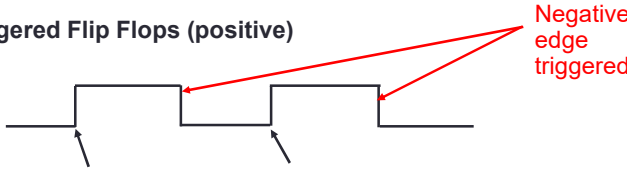
- State transition occurs at the rising edge or falling edge of the clock pulse

Level-Triggered Flip Flops (Latches)



respond to the input only during these periods

Edge-triggered Flip Flops (positive)



respond to the input only at this time (positive)

Negative edge triggered

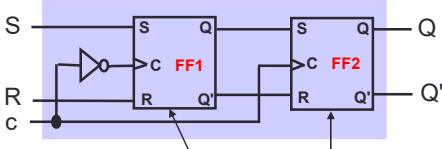
(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

36

Positive Edge-Triggered SR-FF and JK-FF

Edge-Triggered SR Flip Flop



FF1 active

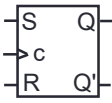
FF2 inactive

FF1 inactive

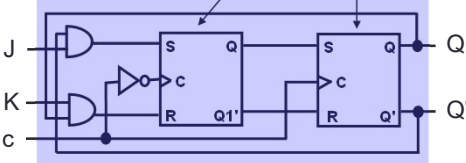
FF2 active

FF1 active

FF2 inactive



Edge-triggered JK Flip Flop



FF1 active

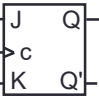
FF2 inactive

FF1 inactive

FF2 active

FF1 active

FF2 inactive



Registers

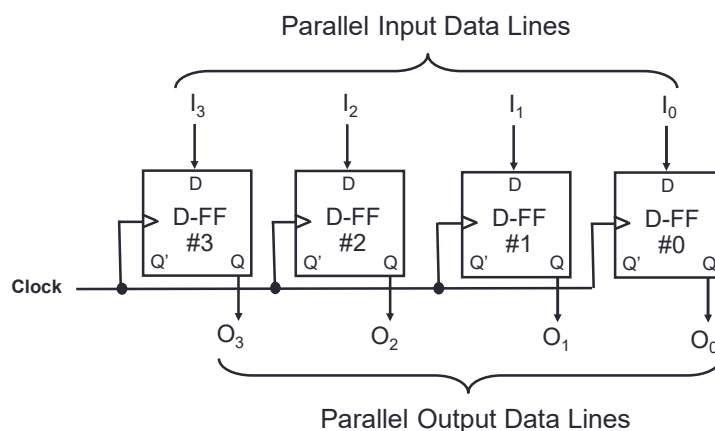
An n-bit register comprises of n flip flops (normally D-FF) with common control and clock lines.
Usually used inside CPUs for fast data storage and retrieval.

Registers can be categorized in 4 classes (according to the storage and retrieval arrangement):

- I. Parallel In Parallel Out (PIPO) Registers:
Data written into FFs in parallel and FFs' contents are read in parallel.
- II. Parallel In Serial Out (PISO) Registers:
Data written into FFs in parallel but FFs' contents are accessed serially.
- III. Serial In Parallel Out (SIPO) Registers:
Data written into FFs serially but FFs' contents are accessed in parallel.
- IV. Serial In Serial Out (SISO) Registers:
Data written into FFs serially and FFs' contents are accessed serially.

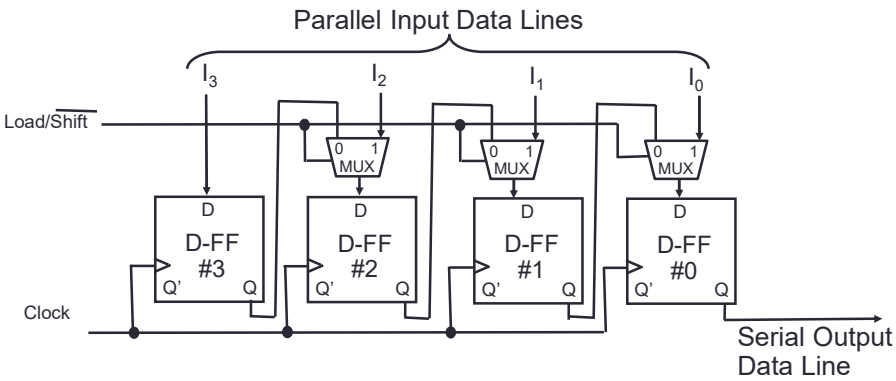
Registers

Example: Draw the logic diagram of a 4-bit PIPO Register.



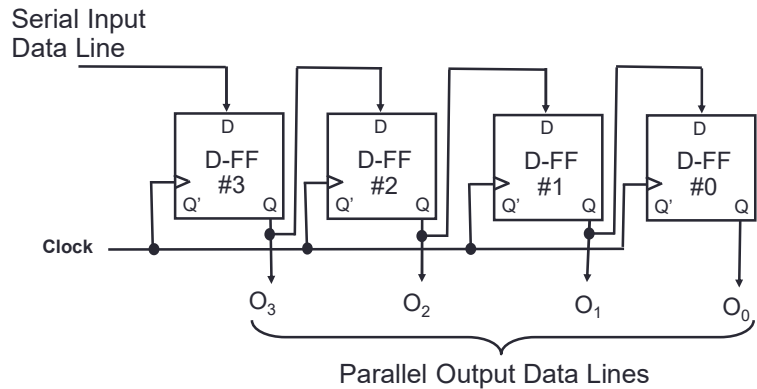
Registers

Example: Draw the logic diagram of a 4-bit PISO (right-shift) Register.



Registers

Example: Draw the logic diagram of a 4-bit SIPO (right-shift) Register.



(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #2: Overview of Logic Circuits41

Registers

Example: Draw the logic diagram of a 4-bit SISO Register (right-shift register).

Serial Input Data Line

Clock

Serial Output Data Line

(c) Hamid Sarbazi-AzadComputer Architecture -- Lecture #2: Overview of Logic Circuits42

Registers

Example: Design a 4-bit general purpose register. It is a PIPO register that can act as a right/left shift register based on the 2-bit command code xy .

x	y	Action
0	0	Nothing
0	1	Shift left
1	0	Shift right
1	1	Load

Parallel Input Data Lines

Serial Right Input Data Line

Serial Left Input Data Line

Serial Left Output Data Line

Clock

Parallel Output Data Lines

Serial Right Output Data Line

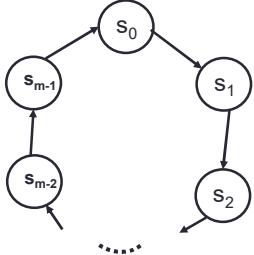
(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

43

Counters

A modulo-m counter start from an state s_0 and passes states $s_1, s_2, \dots$, and s_{m-1} , in order and return to state s_0 .



Counters can be **Synchronous** or **Asynchronous**. In **Synchronous** counters all flip flops are clocked by a **common clock** input.
→ **Expensive but critical for reliable system design**

In **Asynchronous** counters each flip flops has its **separate clock** not necessarily arriving at the same time. → **Cheaper**

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

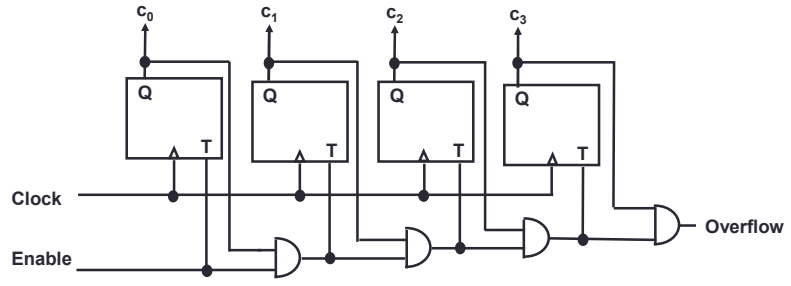
44

Binary Counters

In an n-bit binary **up counter** counting is started from state $s_{00\dots00}$, and then $s_{00\dots01}$, $s_{00\dots10}$, $s_{00\dots11}$, $\dots$, $s_{01\dots11}$, $\dots$, and $s_{11\dots11}$ and then returns to state $s_{00\dots00}$.

In a **down counter** flip flops value starts from initial value **11...11** and then decremented by each clock until **00...00**, and then returns to initial value **11...11**.

Example: Design a synchronous 4-bit binary up counter.



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

45

Binary Counters

Example: Design a synchronous 3-bit down counter.

The top diagram shows a 3-bit down counter using three T flip-flops. The outputs are labeled c_0 , c_1 , and c_2 . The clock is connected to the T input of the first flip-flop. The Enable signal is connected to the T inputs of all three flip-flops. The output of the first flip-flop is connected to the T input of the second, and the output of the second is connected to the T input of the third. The output of the third flip-flop is connected to the underflow signal.

The bottom diagram shows a 3-bit down counter using three D flip-flops. The outputs are labeled c_0 , c_1 , and c_2 . The clock is connected to the D input of the first flip-flop. The Enable signal is connected to the D inputs of all three flip-flops. The output of the first flip-flop is connected to the D input of the second, and the output of the second is connected to the D input of the third. The output of the third flip-flop is connected to the underflow signal.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

46

Binary Counters

Example: Design a synchronous 4-bit up/down counter.

The diagram shows a 4-bit up/down counter using four D flip-flops and four 2-to-1 multiplexers. The outputs are labeled c_0 , c_1 , c_2 , and c_3 . The Up/Down signal is connected to the select input of each multiplexer. The clock is connected to the D input of the first flip-flop. The Enable signal is connected to the D inputs of all four flip-flops. The output of the first flip-flop is connected to the D input of the second, and the output of the second is connected to the D input of the third, and the output of the third is connected to the D input of the fourth. The output of the fourth flip-flop is connected to the carry signal.

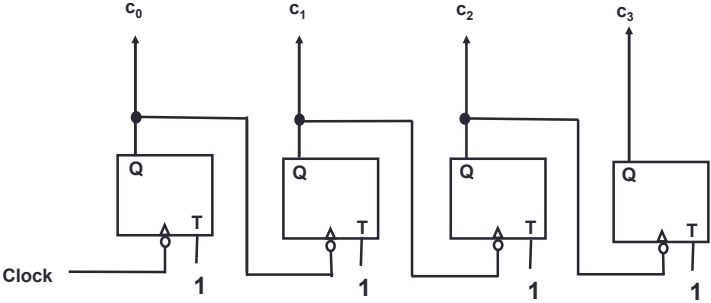
(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

47

Binary Counters

Example: Design an asynchronous 4-bit counter.



c_3

c_2

c_1

c_0

0000

0001

0010

0011

0100

0101

0110

0111

1000

1001

1010

1011

1100

1101

1110

1111

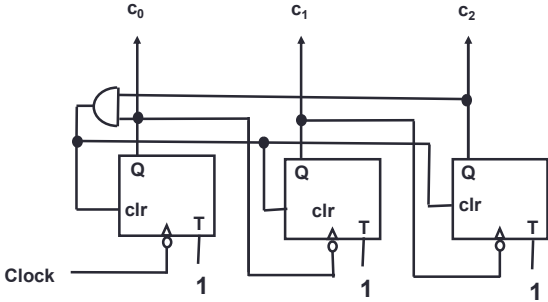
(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

48

Binary Counters

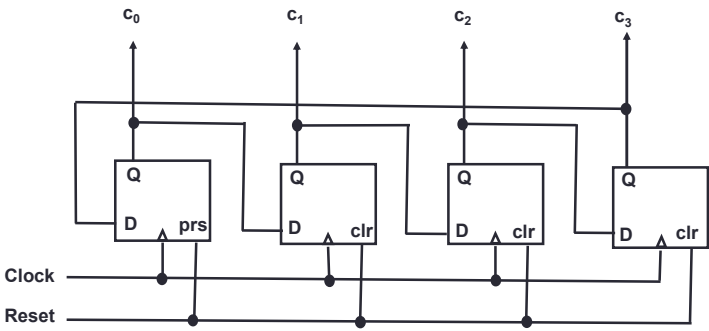
Example: Design a modulo-5 asynchronous 3-bit counter.



Other Counters

An n -bit Ring counter has n states. It starts with initial state $10...00$ and then passes states $010...00$, $0010...00$, ... , $00...010$ and $00...01$, and then return to the initial state $10...00$.

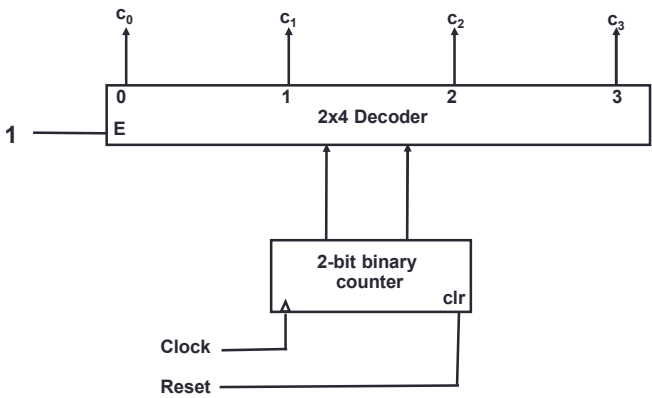
Example: Design a 4-bit Ring counter.



Other Counters

An n -bit Ring counter has n states. It starts with initial state $10...00$ and then passes states $010...00$, $0010...00$, ... , $00...010$ and $00...01$, and then return to the initial state $10...00$.

Example: Design a 4-bit Ring counter. (Alternative Design)



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

51

Other Counters

An n -bit Johnson counter has $2n$ states and starts with initial state $00\dots00$ and then passes states $10\dots00$, $110\dots00$, $1110\dots0$, $\dots$, $11\dots10$, $11\dots11$, $01\dots11$, $001\dots11$, $00\dots11$, $00\dots01$, and then returns to initial state $00\dots00$.

Example: Design a 4-bit Johnson counter.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

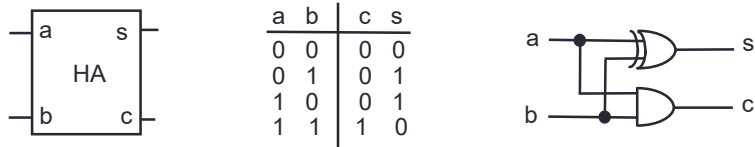
52

Counters

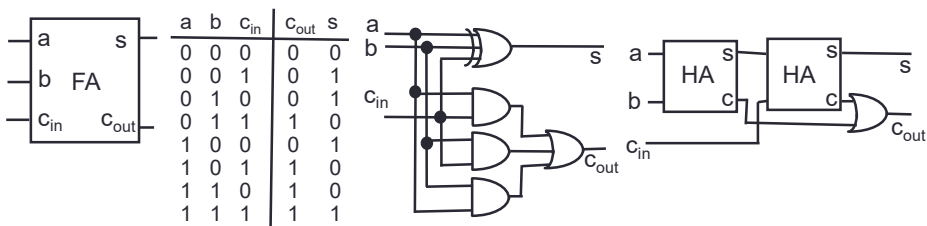
Example: Design an n -bit adder using counters, i.e. $A \leftarrow A + B$;

Half-Adder and Full-Adder

A Half-Adder (HA) adds two bits *a* and *b* and generates 2 bits *s* (sum) and *c* (carry).

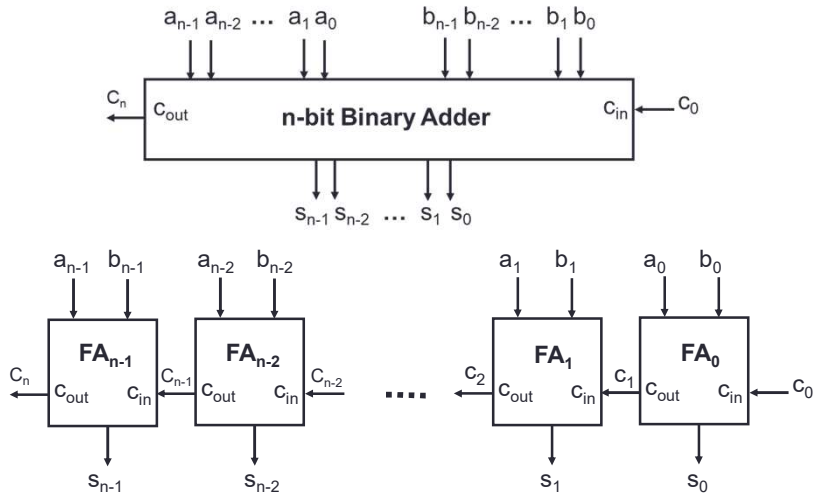


A Full-Adder (FA) adds three bits *a*, *b* and *c* (also named *c_{in}* to indicate input-carry for multibit addition) and generates 2 bits *s* (sum) and *c* (also named *c_{out}*).



Binary Adder

An *n*-bit binary adder can be easily implemented with *n* cascaded FAs.
We add $A = a_{n-1}a_{n-2} \dots a_1a_0$ and $B = b_{n-1}b_{n-2} \dots b_1b_0$ and generate $S = s_{n-1}s_{n-2} \dots s_1s_0$.



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

55

Data Path Inside ALU

Example: Design a simple ALU that can execute 8-bit instructions in a CPU with four 16-bit registers (R0, R1, R2 and R3). Field c_1c_0 shows the operation type, field d_1d_0 shows destination register, and fields s_1s_0 and t_1t_0 show the two source registers.

c_1	c_0	d_1	d_0	s_1	s_0	t_1	t_0
0	0	$d \leftarrow s + t$					
0	1	$d \leftarrow s \wedge t$					
1	0	$d \leftarrow s \vee t$					
1	1	$d \leftarrow s \oplus t$					

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #2: Overview of Logic Circuits

56

Integrated Circuits

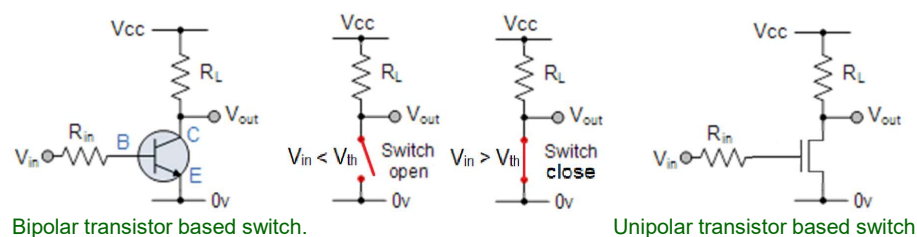
Classification by the Circuit Density:

- SSI** Several (less than 10) independent gates
- MSI** 10 to 200 gates
Perform elementary digital functions: Decoder, adder, register, parity checker, etc.
- LSI** Hundreds to few thousand gates
Digital subsystem: Tiny processor, memory, etc.
- VLSI** Thousands of gates
Digital systems: Microprocessor, memory module
- ULSI** Millions of Gates
System-on-chip: Multicore processors with L1 and L2 caches
- WSI** Billions of Gates
Giant Computing Systems: DNN accelerators

Integrated Circuits

Classification by Technology:

TTL	Transistor-Transistor Logic (Bipolar transistors)
ECL	Emitter-coupled Logic (Bipolar transistors) High current
MOS	Metal-Oxide Semiconductor (Unipolar transistors) High density
CMOS	Complementary MOS Low power consumption



END OF SLIDES

QUESTIONS?